

What routing does to Kimi's memory traffic

Kimi-Linear-48B-A3B (FP8) on the DGX Spark (GB10) · 22 runs, 2026-10-06 · sources: docs/07-moe-routing.md §5, runs/moe-queue/results.md, findings.md

Key takeaways

- Kimi's own router reuses a few favourite experts, so it lands only about **a third of the way** from the best case to the worst case.
- Decode is limited by **kernel launches, not memory bandwidth**: the GPU sits idle 46–69% of every step.
- Each extra expert a step touches costs a fixed **+0.35 GB of L2 traffic and ~9 ms**, whatever the batch size.

What was tested

Kimi has 256 *experts* (small sub-networks) in each of its 26 MoE layers, and every token is sent to 8 of them. When tokens in the same pass share an expert, its weights are read only once, so the memory a pass needs depends on how many *different* experts it touches. We forced three routing patterns to measure the range:

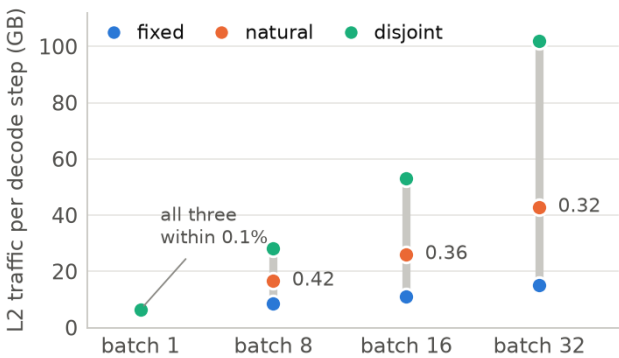
	● fixed	● natural	● disjoint
What happens	Every token forced to experts 0–7	The model's own router chooses	No two tokens share an expert
Experts per step	Always 8	In between	8 × batch, up to 256
Role	Floor (best case)	Realistic case	Ceiling (worst case)

Decode sweep: batch 1, 8, 16 and 32 in each mode (12 runs; prompt 128 tokens, 32 new tokens). **Prefill:** prompts of 16, 64 and 512 tokens at batch 1 (9 runs), plus one 16,384-token long-context run. Bytes are *L2 traffic* (L2 = the GPU's 25 MB cache).

Decode: natural routing lands near the floor

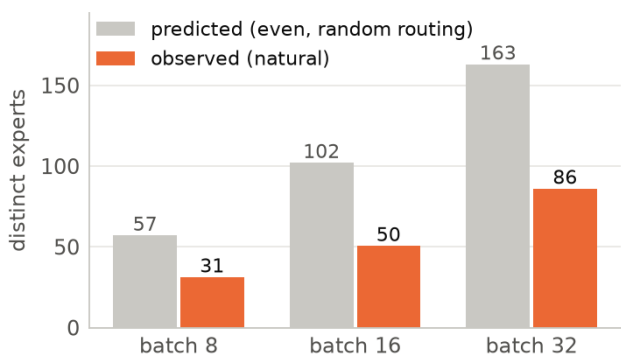
Natural sits nearer the floor than the ceiling

Label = natural's position between fixed (0) and disjoint (1)



Router uses ~half the experts chance predicts

Natural routing, distinct experts per layer per step

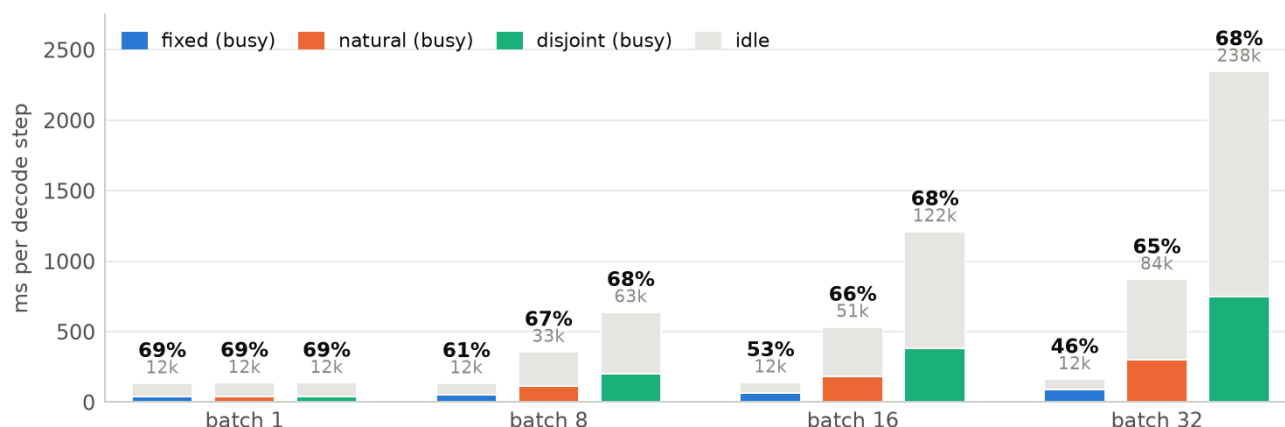


Batch 1 is a self-check: one token always needs 8 experts, and experts don't survive in the 25 MB L2 between steps, so all three modes read the same 6.32 GB. **From batch 8 up the modes separate.** At batch 32, natural reads 43.0 GB per step against 15.2 GB (fixed) and 102.1 GB (disjoint), 0.32 of the way up. The router favours a few experts and uses about half as many as even, random routing would.

What limits decode: launches, not bandwidth

The GPU is idle 46-69% of every decode step

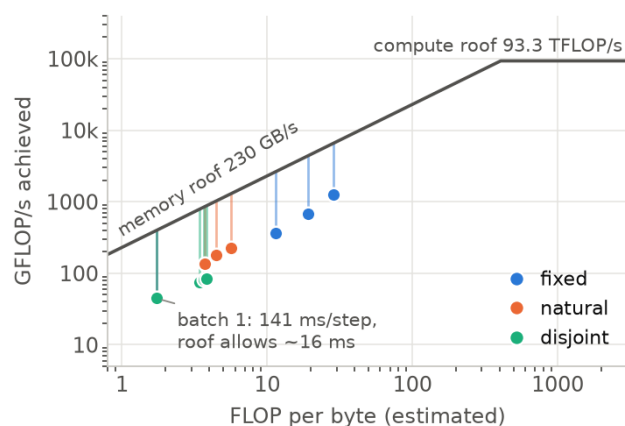
Coloured = GPU busy · grey = GPU idle · above each bar: % of the step idle (bold), kernel launches per step (grey)



The GPU spends most of each step waiting. In every decode run it is busy for only 31-54% of the step and idle for the remaining 46-69%. Kimi's transformers code (*moe_infer*) loops over the used experts one by one in Python and syncs with the CPU in every MoE layer, so every extra expert adds launches: about 12k per step at 8 experts, rising to 238k at disjoint batch 32. The idle share is lowest for fixed routing at large batch (46% at batch 32), where few experts do the most work per launch.

Every decode run sits far under its roof

Vertical line = gap to what bandwidth alone allows



How to read the roofline

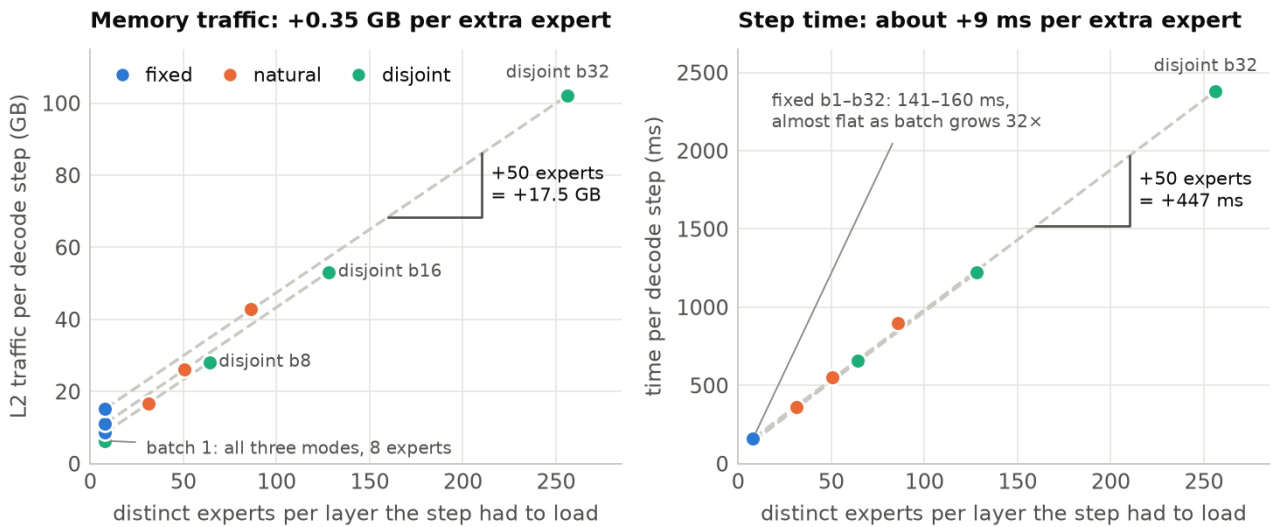
Each dot is one decode run. The sloped line is the most work memory bandwidth could feed; the flat line is the compute limit. A run limited by bandwidth would sit *on* the sloped line.

Instead every run sits **5-10x below** it. A batch-1 step needs about 3.6 GB of weights: ~16 ms at 230 GB/s, yet it takes 141 ms. All runs are also far left of the 405 FLOP/byte ridge, so compute is not the limit either. What is left is launch and sync overhead.

Roofline caveat: FLOPs are estimated ($2 \times \sim 3.1B$ active parameters $\times$ batch; no run measured FLOPs). Bytes are the expected weight bytes, not measured DRAM. Measured L2 traffic is $\sim 1.9\times$ this, and DRAM was never measured on Kimi. The gap is large enough that the conclusion holds either way.

Every extra expert costs the same, at every batch

Each dot is one decode run, placed by how many different experts each layer had to load for a step: fixed is always at 8, disjoint at $8 \times \text{batch}$, natural wherever its router landed (counted by the routing observer). A dashed line joins fixed and disjoint at the same batch.

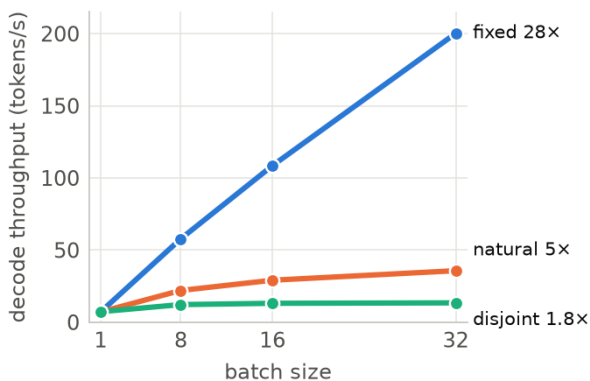


What the chart shows

- 1. Straight lines with one slope.** Going from fixed to disjoint, every extra expert adds **0.35 GB** of L2 traffic at batch 8, 16 and 32 alike, and about **9 ms** (9.3 / 8.9 / 8.9 ms). Natural routing lands on the same lines. On the time panel the three lines lie on top of each other.
- 2. Two independent measurements agree.** Reading natural's expert count off the L2 line gives 31.5 / 50.9 / 87.2; the routing observer counted 31.2 / 50.5 / 86.0. Predictions made from this line before each run landed within 0.02–1.6%.
- 3. Tokens are nearly free; experts are not.** At 8 experts (fixed), going from batch 1 to 32 multiplies the tokens per step by 32 but only moves step time from 141 to 160 ms. L2 traffic rises from 6.3 to 15.2 GB, which is why the L2 lines are offset upward with batch. That is small next to the experts: the 248 extra experts of disjoint batch 32 add ~87 GB and ~2.2 s.
- 4. Each expert moves ~2× its weight size through L2:** ~0.18 GB of weights (all 26 layers) but 0.35 GB of L2 traffic. Compare these GB between runs, not as memory reads.

What batching buys depends on sharing

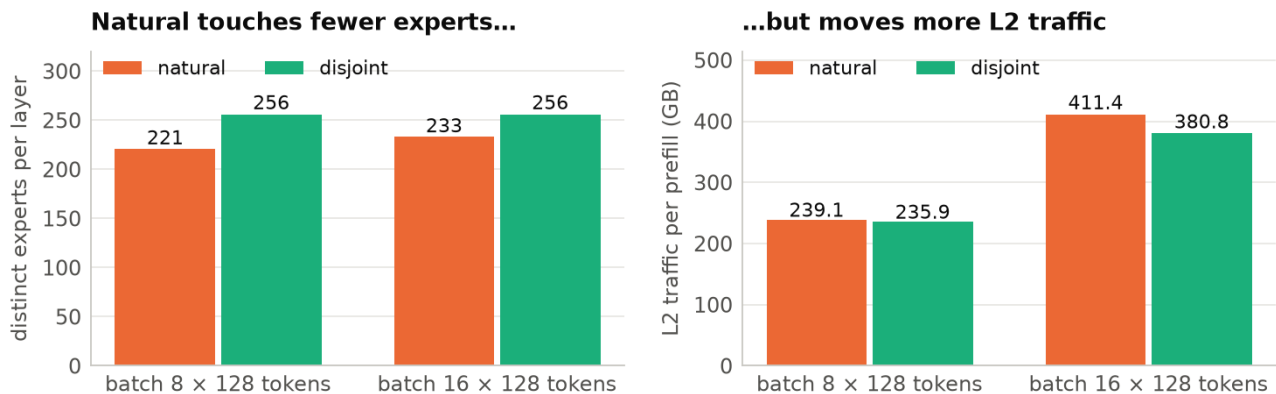
Throughput gain, batch 1 → 32, labelled at right



So what: batching pays off only when tokens share experts

Because cost follows experts, the gain from batching depends on how much the batch's tokens share. From batch 1 to 32, throughput rises **28×** under fixed routing (step time almost flat), **5×** under natural, and only **1.8×** under disjoint. Under natural routing, most of batching's potential is lost to the ~9 ms each extra expert costs in Kimi's per-expert kernel launches.

The prefill anomaly: fewer experts, more L2 traffic



In the two largest batched prefills (1,024 and 2,048 tokens in one pass), natural routing used **fewer** experts than disjoint but moved **more** L2 traffic: 3 GB more at batch 8, 31 GB more at batch 16. The 0.35 GB-per-expert rule predicts it should be 8–12 GB *below* disjoint.

Why: natural routing is very uneven. Each layer's 8 most popular experts receive **~44%** of all token picks, against 3% if routing were even, so a popular expert gets ~450 tokens while a disjoint expert gets ~32. The GPU works through an expert in tiles of about 128 tokens, and **each tile re-reads the expert's weights through L2**. A popular expert spanning 4–7 tiles is therefore read 4–7 times; a disjoint expert fits in one tile and is read once. The extra re-reads outweigh natural's ~35 fewer experts, and the gap grows at batch 16 because popular experts get twice the tokens.

What it means: one expert's weights (~7 MB per layer) fit in the 25 MB L2, so these re-reads are most likely cache hits. Natural routing probably still reads *less* main memory than disjoint, as its expert count suggests (not verified). Natural is still the faster of the two (2.30 s vs 2.56 s at batch 8). Decode is unaffected, since at most 32 tokens per step means every expert fits in one tile.

Caveats

1. **Bytes are L2 traffic, not DRAM.** On Kimi, L2 runs ~1.9× the weight bytes a step needs. No exact-DRAM measurement exists on Kimi; the L2-vs-DRAM check was done on Qwen3-0.6B only. Compare GB between runs, not as absolute memory reads.
2. **The roofline's FLOPs are estimated,** not measured (ncu tier 1 was off because it takes ~13 h per Kimi run).
3. **Prompts were synthetic random token IDs.** Natural routing on real text may spread tokens differently, so natural's position between the bounds could move.
4. **The launch-bound result reflects the transformers implementation** (per-expert Python loop, CPU sync in every MoE layer), not Kimi's architecture as such. Other inference engines were not tested.
5. **The prefill mechanism is confirmed only on early-layer GEMMs.** The ncu sample covered the first ~4–5 linear layers and never reached the routed experts (same kernel family). No routing conclusion here depends on ncu.
6. **For large prefills, L2 GB do not rank routing modes reliably.** Use expert counts and time there.